Tugas Besar
Milestone 1

IF2224 Teori Bahasa Formal dan Otomata

Lexical Analysis

Semester 2 Tahun 2025/2026

Disusun oleh:

Kai

Jonathan Kris Wicaksono	(13524023)
Vincent Rionarlie	(13524031)
Muhammad Aufar Rizqi Kusuam	(13524061)
Bryan Pratama Putra Hendra	(13524067)

Laboratorium Ilmu dan Rekayasa Komputasi
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Daftar Isi

1 Pendahuluan **3**
1.1 Pengantar . . . 3
1.2 Spesifikasi Program . . . 4
1.2.1 Daftar Token Bahasa Pemrograman Arion . . . 5
1.2.2 Contoh Input/Output . . . 6

2 Dasar Teori **9**
2.1 Deterministic Finite Automata (DFA) . . . 9
2.1.1 Intuisi dan Definisi Formal . . . 9
2.1.2 Komponen dan Aplikasi DFA . . . 10
2.1.3 Contoh DFA dan Tabel Transisi . . . 10
2.2 Token . . . 12
2.2.1 Definisi dan Struktur Token . . . 12
2.2.2 Kategori Token . . . 12
2.2.3 Peran Token pada Proses Kompilasi . . . 12
2.3 Lexical Analysis . . . 13
2.3.1 Definisi dan Tujuan . . . 13
2.3.2 Peran Lexer/Scanner . . . 13
2.3.3 Alur Kerja Lexical Analysis . . . 13
2.3.4 Penanganan Simbol Non-Esensial . . . 14
2.3.5 Keluaran Token Stream . . . 14

3 Perancangan dan Implementasi **15**
3.1 Diagram Transisi DFA Program . . . 15
3.2 Gambaran Umum Program . . . 16
3.3 Algoritma Lexer . . . 17
3.4 Struktur Direktori Program . . . 20
3.5 Implementasi Kode . . . 21
3.5.1 `token.h` . . . 21
3.5.2 `token.cpp` . . . 22
3.5.3 `lexer.h` . . . 23
3.5.4 `lexer.cpp` . . . 24

4 Pengujian **27**
4.1 Kasus Dasar . . . 27
4.2 *Edge Cases* . . . 27

5 Penutup **29**
5.1 Kesimpulan 29
5.2 Saran 29

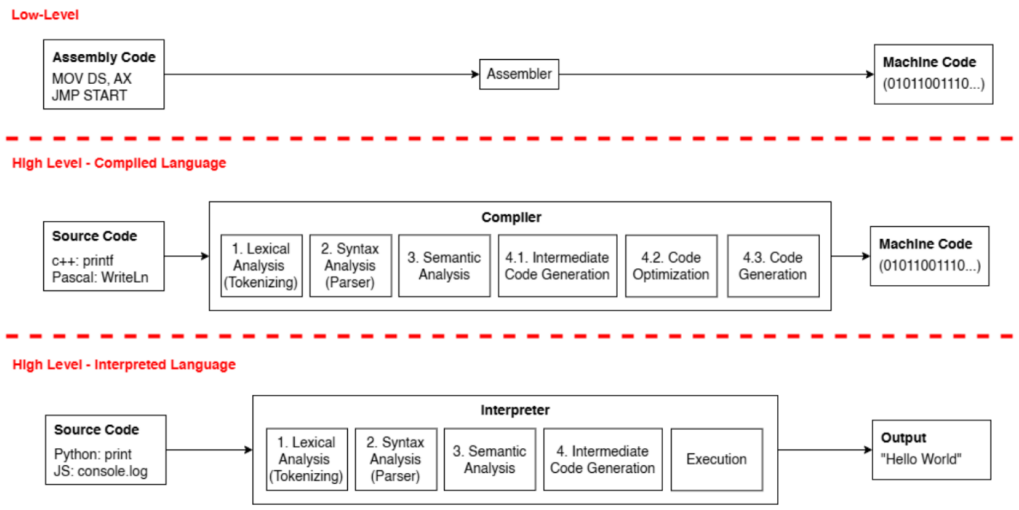
6 Lampiran **30**
6.1 Tautan GitHub 30
6.2 Tautan Diagram Transisi DFA 30
6.3 Tabel Kontribusi 30

Bab 1

Pendahuluan

1.1 Pengantar

Apa itu **Bahasa Pemrograman?** Bahasa pemrograman adalah cara agar kita dapat menuliskan program komputer tanpa perlu menghadapi sulitnya menulis dalam bahasa mesin. Bahasa mesin (biner 1 dan 0) adalah satu-satunya bahasa yang dapat dipahami dan dijalankan oleh CPU, namun sangat sulit dibaca oleh manusia. Oleh karena itu, bahasa pemrograman sering kali dibedakan dengan istilah low-level atau high-level. High-level artinya bahasa lebih mendekati bahasa manusia dan sebaliknya low-level lebih mendekati bahasa mesin. Semakin mendekati bahasa manusia, maka akan semakin banyak proses yang diperlukan untuk mengubahnya menjadi kode mesin.



Gambar 1.1: Tipe-tipe bahasa pemrograman (Sumber: Asisten IF2224)

Terdapat dua macam cara mengubah/memproses bahasa pemrograman menjadi machine code, yaitu **compiler** dan **interpreter**. Compiler adalah program yang menerjemahkan seluruh source code menjadi kode mesin (dalam bentuk file e.g. Object File) sebelum program dijalankan. Compiler secara umum terdiri dari 4 tahapan, mulai dari Lexical Analysis, Syntax Analysis, Semantic Analysis hingga Intermediate Code Generation.

Di sisi lain adalah Interpreter yang memproses source code baris per baris secara langsung. Interpreter membaca, menerjemahkan, dan langsung mengeksekusi kode mesin baris per baris sehingga jika terdapat error pada baris ke 10, program akan tetap berjalan normal untuk 9 baris awal, dibandingkan dengan compiler yang akan gagal dijalankan jika terdeteksi terdapat error di dalam source code. Namun dibalik itu, interpreter sebenarnya memiliki tahapan yang sama seperti compiler.

Pada Tugas Besar Teori Bahasa Formal dan Automata, Mahasiswa diminta untuk membuat Compiler beserta Interpreter untuk bahasa pemrograman *Arion*, suatu bahasa pemrograman baru yang akan Anda kembangkan. Tugas besar ini dibagi menjadi 4 Milestone, yaitu Lexical Analysis, Syntax Analysis, Semantic Analysis, Intermediate Code Generation dan Interpreter. Setiap Milestone akan memiliki pengumpulannya dan penilaiannya tersendiri.

Pada **milestone pertama**, Mahasiswa diminta untuk membuat tahapan Lexical Analysis dari suatu compiler.

1.2 Spesifikasi Program

Pada milestone 1, Mahasiswa diminta untuk membuat **lexer** atau **lexical analyzer** dalam C/C++ yang menggunakan Deterministic Finite Automata (DFA) untuk menghasilkan token dari source code bahasa pemrograman Arion.

Dalam milestone ini, Mahasiswa diminta untuk membuat program Lexer dalam bahasa pemrograman C/C++ GNU. Mahasiswa **DILARANG** menggunakan library atau tools apapun yang dapat membantu menghasilkan lexical analyzer. Anda wajib menggunakan **MAKEFILE** untuk mempermudah pengembangan dalam bahasa C/C++.

Program Lexer harus berfungsi selayaknya suatu DFA. Program Lexer **HANYA BOLEH** membaca satu demi satu karakter dari input source code dan memprosesnya sesuai dengan aturan pada DFA. Program harus dapat menerima input source code dalam bentuk `.txt`. Program kemudian memproses source code dengan aturan DFA dan mengeluarkan output `daftar token` dalam format `.txt`.

1. Masukan : Kode Sumber (`.txt`)
2. Keluaran : Daftar Token (`.txt`)
3. Otomata : Deterministic Finite Automata

Catatan: Untuk mempermudah pengerjaan pada milestone-milestone selanjutnya, khususnya pada milestone 4 (pembuatan interpreter), SANGAT DISARANKAN agar program dikembangkan secara modular (terpisah antara komponen Lexer, Semantic, Syntax, dan Intermediate Code Generation). Hal ini supaya komponen dapat dimodifikasi sesuai keperluan tanpa mempengaruhi komponen lainnya, serta dapat dengan mudah dijalankan dalam satu pipeline: lexer - syntax - semantic - intermediate code generation.

1.2.1 Daftar Token Bahasa Pemrograman Arion

Lexer bertujuan untuk mengubah source code dari kumpulan karakter mentah menjadi unit-unit bermakna yang disebut token. Setiap token memiliki jenis (type) dan nilai (value) yang akan digunakan untuk tahap selanjutnya dalam proses kompilasi. Lexer diharapkan dapat menerima source code dan mengeluarkan daftar token yang dihasilkan. Berikut merupakan daftar token yang harus dapat dibaca oleh lexical analyzer.

Tabel 1.1: Daftar token bahasa Arion

No	Token	Definisi	Contoh Pemakaian
1	intcon	Konstanta integer	`1, 3, 48`
2	realcon	Konstanta bilangan riil	`3.14, 26.7`
3	charcon	Konstanta karakter singular, diapit dengan tanda petik tunggal	`'j', 'k', 't'`
4	string	Sekuens karakter, diapit dengan tanda petik tunggal	`'IRK', 'TBFO'`
5	notsy	Operator logika NOT (negasi)	`NOT`
6	plus	Operator aritmatika pertambahan	`+`
7	minus	Operator aritmatika pengurangan	`-`
8	times	Operator aritmatika perkalian	`*`
9	idiv	Operator aritmatika pembagian Integer	`div`
10	rdiv	Operator aritmatika pembagian Riil	`/`
11	imod	Operator aritmatika modulo	`MOD`
12	andsy	Operator logika AND	`AND`
13	orsy	Operator logika OR	`OR`
14	eql	equal	`==`
15	neq	Not equal	`<>`
16	gtr	Greater than	`>`
17	geq	Greater than or equal	`>=`
18	lss	Less than	`<`
19	leq	Less than or equal	`<=`
20	lparent	Left parentheses	`(`
21	rparent	Right parantheses	`)`
22	lbrack	Kurung siku kiri	`[`
23	rbrack	Kurung siku kanan	`]`
24	comma	Koma	`,`
25	semicolon	Titik koma	`;`
26	period	Titik	`.`
27	colon	Titik dua	`:`
28	becomes	Assignment operator	`:=`

No	Token	Definisi	Contoh Pemakaian
29	constsy	Deklarasi konstanta	`const`
30	typesy	Deklarasi tipe data	`type`
31	varsy	Deklarasi variabel	`var`
32	functionsy	Deklarasi fungsi	`function`
33	proceduresy	Deklarasi prosedur	`procedure`
34	arraysy	Deklarasi array	`array`
35	recordsy	Deklarasi record	`record`
36	programsy	Deklarasi program	`program`
37	ident	Nama yang menunjukkan konstanta, tipe, variabel, prosedur, dan fungsi; diawali huruf, diikuti huruf/angka; case-insensitive	`x, PI, MyInt, h4g`
38	beginsy	Keyword begin	`begin`
39	ifsy	Keyword if	`if`
40	casesy	Keyword case	`case`
41	repeatsy	Keyword repeat	`repeat`
42	whilesy	Keyword while	`while`
43	forsy	Keyword for	`for`
44	endsy	Keyword end	`end`
45	elsesy	Keyword else	`else`
46	untilsy	Keyword until	`until`
47	ofsy	Keyword of	`of`
48	dosy	Keyword do	`do`
49	tosy	Keyword to	`to`
50	downtosy	Keyword downto	`downto`
51	thensy	Keyword then	`then`
52	comment	Dimulai dengan { atau (* (bukan di dalam tanda kutip) dan diakhiri dengan } atau *)	`{ Ini komentar }`

1.2.2 Contoh Input/Output

Input

```
program Hello;

var
    a, b: integer;

begin
```

```
    a := 5;
    b := a + 10;
    writeln('Result = ', b);
end.
```

Visualisasi Pemrosesan (DFA)

```
p => State 1
r => State 1
.
m => State 2 => Gotten: program
.
H => State 3
e
.
; => char => State 0 => Gotten: ident(Hello) & semicolon
.
.
.
```

Output

```
programsy
ident (Hello)
semicolon

varsy
ident (a)
comma
ident (b)
colon
ident (integer)
semicolon

beginsy
ident (a)
becomes
intcon (5)
semicolon
ident (b)
becomes
ident (a)
plus
intcon (10)
semicolon
ident (writeln)
```

```
lparent
string ('Result = ')
comma
ident (b)
rparent
semicolon

endsy
period
```

Bab 2

Dasar Teori

2.1 Deterministic Finite Automata (DFA)

2.1.1 Intuisi dan Definisi Formal

Secara intuitif, *Deterministic Finite Automata* (DFA) dapat dipahami sebagai "mesin pembaca" yang bergerak langkah demi langkah saat menerima karakter masukan. Pada setiap langkah, mesin selalu berada tepat pada satu keadaan (*state*) tertentu, lalu berpindah ke keadaan berikutnya berdasarkan karakter yang sedang dibaca. Sifat paling penting dari DFA adalah deterministik: untuk setiap pasangan keadaan dan simbol masukan, hanya ada satu perpindahan yang mungkin. Karena itu, perilaku DFA dapat diprediksi dan sangat cocok digunakan pada tahap lexical analysis yang menuntut keputusan cepat serta konsisten untuk setiap karakter yang dibaca dari kiri ke kanan [1, 2].

Secara formal, sebuah DFA didefinisikan sebagai *quintuple* di bawah ini

$$A = (Q, \Sigma, \delta, q_0, F).$$

Himpunan berhingga Q menyatakan semua state yang mungkin, Σ adalah himpunan simbol masukan, $\delta : Q \times \Sigma \to Q$ adalah fungsi transisi yang memetakan pasangan state beserta sebuah simbol masukan ke sebuah state lain, $q_0 \in Q$ adalah state awal dari mesin, dan $F \subseteq Q$ adalah himpunan *acceptance state*. Untuk sebuah string masukan w, DFA memproses simbol demi simbol melalui perluasan fungsi transisi $\hat{\delta}$, yaitu string dinyatakan diterima jika state akhir pemrosesan berada di dalam F. Formulasi ini memberi dasar matematis yang kuat untuk menyatakan apakah suatu pola leksikal termasuk bahasa yang diinginkan atau tidak [1, 2].

Lewat quintuple ini, dapat dimodelkan sebuah mesin yang dapat menerima sejumlah input karakter Σ, dengan state awal dari mesin adalah q_0, yang memiliki sejumlah state mesin disimpan dalam Q dan dapat berpindah-pindah dari satu state ke state yang lain menggunakan fungsi transisi δ. Sejumlah input karakter merupakan sebuah "bahasa" yang valid jika state akhir dari mesin tersebut berada pada himpunan state final F.

2.1.2 Komponen dan Aplikasi DFA

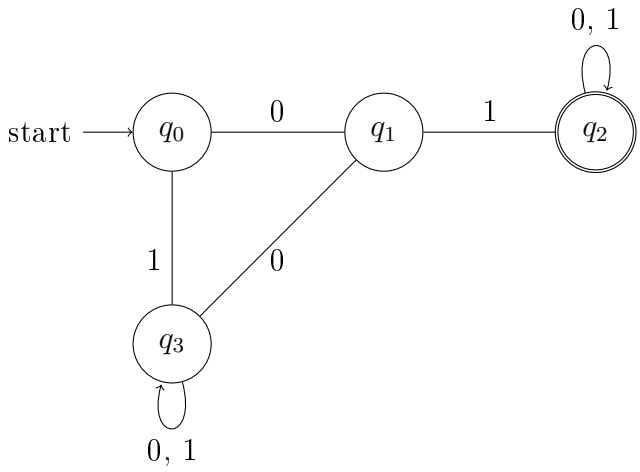
Komponen pembangun DFA pada implementasi lexer dapat dipetakan langsung dari definisi formal tersebut. State merepresentasikan konteks pembacaan token, misalnya state awal, state saat membaca identifier, state saat membaca bilangan, atau state saat berada di dalam komentar. Alfabet masukan berupa karakter-karakter source code yang relevan (huruf, digit, operator, delimiter, dan simbol khusus lain). Fungsi transisi diwujudkan sebagai aturan *if/switch* yang menentukan state berikutnya untuk setiap karakter yang masuk. State awal dipakai ketika belum ada token yang sedang dibangun, sedangkan state penerima menandai bahwa pola token valid telah lengkap dan token siap di-*emit*. Dengan pemetaan ini, desain DFA tidak hanya teoretis, tetapi langsung menjadi kerangka arsitektur program lexer yang modular dan mudah diuji [3, 4].

Aplikasi DFA sangat luas pada perangkat lunak sistem, dan pada konteks tugas besar ini aplikasi utamanya adalah lexical analyzer/compiler front-end. DFA dipakai untuk mengenali lexeme seperti keyword, identifier, integer, real, operator, dan komentar secara efisien dalam waktu linear terhadap panjang input. Di luar compiler, konsep yang sama juga digunakan pada mesin pencocokan pola berbasis regular expression, penyaring teks, validasi format masukan, hingga sebagian mekanisme inspeksi paket pada jaringan yang membutuhkan klasifikasi string secara cepat dan deterministik. Karena kesederhanaan model serta jaminan performanya, DFA menjadi fondasi praktis yang menjembatani teori bahasa formal dengan implementasi perangkat lunak nyata [3, 1].

2.1.3 Contoh DFA dan Tabel Transisi

Sebagai ilustrasi konkret, perhatikan DFA sederhana yang dirancang untuk menerima semua string yang dimulai dengan "01" (atas alfabet biner $\{0, 1\}$). DFA ini memiliki empat state: state awal q_0, state setelah membaca '0' yaitu q_1, state penerima q_2 yang dicapai setelah membaca "01", dan *dead state* q_3 yang merepresentasikan penolakan (karena pola tidak cocok).

Diagram transisi DFA tersebut dapat divisualisasikan sebagai berikut.



Gambar 2.1: Diagram transisi DFA untuk menerima string dimulai dengan "01" (dengan dead state)

Diagram transisi di atas juga dapat direpresentasikan dengan menggunakan sebuah tabel transisi. Semua kombinasi state dan simbol masukan memiliki transisi yang jelas (sesuai dengan konsep DFA itu sendiri). Berikut adalah tabelnya.

State	**0**	**1**
$\rightarrow q_0$	q_1	q_3
q_1	q_3	q_2
$q_2\star$	q_2	q_2
q_3	q_3	q_3

Tabel 2.1: Tabel transisi DFA untuk string dimulai dengan "01"

Dead state (atau *reject state*) adalah state yang bukan penerima dan dari mana mesin tidak dapat keluar menuju state penerima. Setelah mesin memasuki dead state, ia akan tetap berada di state tersebut untuk semua input yang tersisa, sehingga string otomatis ditolak. Dalam kasus ini, q_3 merupakan sebuah dead state. Sekali mesin tersebut masuk ke dalam state q_3, maka apapun masukan selanjutnya akan tetap membuat mesin berada pada q_3. Penggunaan dead state penting untuk menjamin bahwa fungsi transisi δ selalu terdefinisi untuk setiap kombinasi state dan simbol masukan, sesuai dengan definisi formal DFA.

Proses penerimaan string dapat diikuti langkah demi langkah. Sebagai contoh, Untuk string masukan "`0110`": dimulai dari q_0, karakter '0' membawa ke q_1; karakter '1' membawa ke q_2 (state penerima); karakter '1' tetap di q_2; karakter '0' tetap di q_2. Pada akhir string, mesin berada di state penerima q_2, sehingga "`0110`" diterima.

Contoh yang lain, Untuk string "`10`" yang dimulai dengan '1', mesin membaca '1' dari q_0 dan berpindah ke dead state q_3; kemudian membaca '0' dari q_3 dan tetap di q_3. Pada akhir string, mesin berada di dead state q_3, sehingga string ditolak.

2.2 Token

2.2.1 Definisi dan Struktur Token

Token adalah unit terkecil dari source code yang bermakna dan dapat dikenali oleh lexer. Dalam konteks analisis leksikal, *token* berfungsi sebagai jembatan antara rangkaian karakter mentah (source code) dan struktur sintaksis yang akan diproses oleh parser pada tahap berikutnya. Setiap token terdiri dari dua komponen utama: tipe token (*token type*) dan nilai token (*token value*). Tipe token mengidentifikasi kategori semantik dari token tersebut—misalnya apakah token itu keyword, identifier, operator, atau literal konstanta. Nilai token merupakan string spesifik yang dikembalikan dari source code, digunakan khususnya untuk token yang memerlukan informasi detail seperti identifier berupa "x" atau konstanta berupa "42".

Contoh sederhana dari rangkaian token adalah sebagai berikut. Jika source code berisi pernyataan "`x := 10`", lexer akan menghasilkan tiga token: (1) token dengan tipe IDENT dan nilai " x", (2) token dengan tipe BECOMES dan tanpa nilai khusus, dan (3) token dengan tipe INTCON dan nilai "10". Setiap token menyimpan informasi tambahan seperti nomor baris dan kolom tempat token ditemukan, membantu dalam pelaporan error yang akurat jika diperlukan pada tahap parsing atau analisis semantik.

2.2.2 Kategori Token

Klasifikasi token dalam bahasa pemrograman umumnya mencakup beberapa kategori besar. Kategori pertama adalah *keyword* atau kata kunci, yang merupakan simbol-simbol tereservasi dengan makna khusus dalam bahasa, seperti "`if`", "`while`", atau "`function`". Kategori kedua adalah *identifier*, yang merupakan nama-nama yang didefinisikan oleh programmer untuk variabel, fungsi, atau tipe data, dan umumnya bersifat case-insensitive di bahasa tertentu. Kategori ketiga adalah *konstanta atau literal*, yang merepresentasikan nilai tetap seperti bilangan integer ("`42`"), bilangan riil ("`3.14`"), karakter ("`'a'`"), atau string ("`'hello world'`"). Kategori keempat adalah *operator*, yang merupakan simbol untuk operasi aritmatika ("`+`", "`-`"), logika ("`and`", "`or`"), atau perbandingan ("`==`", "`<`"). Kategori kelima adalah *delimiter atau punctuation*, yaitu simbol-simbol struktural seperti tanda kurung, koma, titik koma, dan titik. Terakhir, kategori keenam mencakup *komentar*, yang merupakan teks yang tidak mempengaruhi semantik program dan umumnya dibuang atau disimpan terpisah oleh lexer.

2.2.3 Peran Token pada Proses Kompilasi

Peran token sangat penting dalam pipeline kompilasi modern. Dengan mengubah rangkaian karakter menjadi token, lexer menyederhanakan tugas parser: daripada parser harus menangani jutaan karakter individual, parser bekerja dengan puluhan atau ratusan token yang sudah terklasifikasi. Hal ini meningkatkan efisiensi dan kejelasan dari analisis sintaksis. Lebih lanjut, dengan mendeteksi error leksikal—seperti simbol yang tidak dikenal—pada tahap lexical analysis, proses kompilasi secara kese-

luruhan menjadi lebih robust dan memudahkan programmer untuk mengidentifikasi dan memperbaiki kesalahan [3, 1].

2.3 Lexical Analysis

2.3.1 Definisi dan Tujuan

Analisis leksikal (*lexical analysis*) adalah tahap pertama dalam proses kompilasi atau interpretasi suatu bahasa pemrograman. Pada tahap ini, compiler membaca kode sumber yang pada awalnya hanya berupa rangkaian karakter mentah dan mengubahnya menjadi rangkaian token bermakna. Proses ini sangat krusial karena menjadi pondasi bagi semua tahap analisis berikutnya. Tanpa hasil tokenisasi yang akurat, tahap parsing (syntax analysis) pada langkah berikutnya tidak dapat berjalan dengan baik. Lexical analysis memisahkan tugas pengenalan pola leksikal dasar (karakter demi karakter) dari tugas analisis struktur gramatikal yang lebih kompleks, sehingga meningkatkan modularitas dan maintainability dari compiler secara keseluruhan.

2.3.2 Peran Lexer/Scanner

Komponen utama yang bertanggung jawab atas lexical analysis disebut *lexer* atau *scanner*. Lexer adalah program atau modul yang membaca input source code secara sekuensial dari kiri ke kanan, karakter per karakter, dan mengidentifikasi pola-pola yang sesuai dengan definisi token bahasa yang ditargetkan. Implementasi lexer modern umumnya berbasis DFA atau NFA (Nondeterministic Finite Automata), yang memungkinkan pengenalan pola dengan efisiensi tinggi. Pada setiap tahap pembacaan, lexer menentukan apakah karakter yang sedang dibaca merupakan lanjutan dari token sebelumnya ataukah awal dari token baru. Ketika lexer menemukan bahwa pola karakter telah lengkap dan sesuai dengan definisi token, lexer akan memancarkan (emit) token tersebut ke dalam output token stream.

2.3.3 Alur Kerja Lexical Analysis

Proses lexical analysis secara umum mengikuti alur berikut: pertama, lexer membaca karakter masukan satu per satu menggunakan buffer atau pointer masukan. Kedua, berdasarkan karakter yang dibaca dan state DFA saat ini, lexer melakukan transisi ke state berikutnya atau menyelesaikan pembacaan satu token. Ketiga, ketika lexer mendeteksi batas token (misalnya ketika karakter berikutnya bukan bagian dari token saat ini), lexer melakukan klasifikasi terhadap lexeme yang terkumpul dan memancarkan token yang sesuai. Keempat, lexer kembali ke state awal dan siap membaca lexeme berikutnya. Selama proses ini, lexer juga menangani karakter-karakter non-esensial seperti whitespace (spasi, tab, newline), komentar, dan simbol-simbol lain yang tidak berkontribusi pada semantik program.

2.3.4 Penanganan Simbol Non-Esensial

Pembuangan whitespace dan komentar selama lexical analysis memiliki tujuan penting. Whitespace seperti spasi dan indentasi membantu programmer membaca kode dengan lebih mudah, tetapi tidak mempengaruhi semantik program (kecuali pada bahasa tertentu seperti Python). Komentar adalah teks yang disisipkan programmer untuk dokumentasi dan juga tidak mempengaruhi eksekusi program. Dengan membuang elemen-elemen ini di tahap lexical analysis, lexer mengurangi beban kerja parser dan meningkatkan efisiensi keseluruhan. Namun, informasi baris dan kolom tempat setiap token ditemukan tetap disimpan dalam representasi token, sehingga jika terjadi error di tahap berikutnya, compiler dapat melaporkan lokasi error dengan akurat kepada programmer.

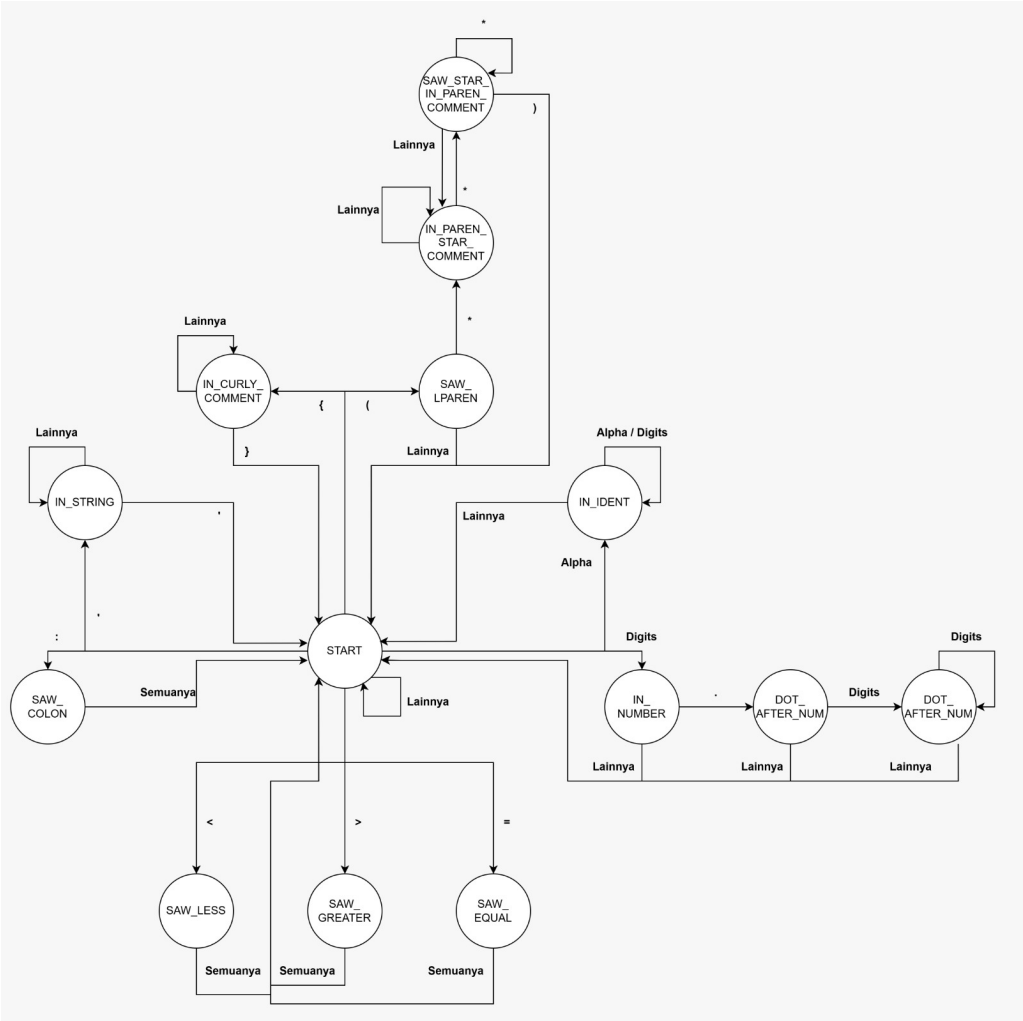
2.3.5 Keluaran Token Stream

Output dari lexical analysis adalah token stream, yaitu urutan token-token yang telah terklasifikasi dan siap untuk diproses oleh parser. Token stream ini biasanya disimpan dalam struktur data seperti array atau linked list, atau dalam beberapa implementasi, di-stream secara on-demand ke parser. Setiap token dalam stream menyimpan informasi lengkap: tipe token, nilai token (jika ada), dan metadata lokasi (baris dan kolom). Format output yang terstandarisasi dan terstruktur ini memastikan bahwa parser dapat fokus pada analisis struktur gramatikal tanpa perlu terganggu oleh detail-detail karakter level yang kompleks [3, 4].

Bab 3

Perancangan dan Implementasi

3.1 Diagram Transisi DFA Program



Gambar 3.1: Diagram Transisi Program (Sumber: Dokumentasi Kelompok)

3.2 Gambaran Umum Program

Program lexer untuk bahasa pemrograman Arion didesain sebagai aplikasi berbasis command-line yang menerima masukan berupa file source code dan menghasilkan daftar token sebagai keluaran. Alur umum eksekusi program adalah sebagai berikut.

1. **Validasi Argumen**. Program menerima dua argumen dari command-line: nama file input (wajib) dan nama file output (opsional). Jika argument tidak sesuai, program akan menampilkan pesan error dan keluar dengan exit code 1.

2. **Pembacaan File Input**. Program membuka file input dengan mode baca. Jika file tidak dapat dibuka, program menampilkan error message dan keluar. Seluruh isi file dibaca ke dalam buffer string di memori.

3. **Inisialisasi Lexer**. Objek atau struktur Lexer diinisialisasi dengan string source code yang sudah dibaca. State awal DFA disetel ke `START`, posisi pembacaan diatur ke 0, dan daftar token diinisialisasi kosong.

4. **Tokenisasi**. Lexer menjalankan mesin DFA dengan membaca karakter demi karakter dari source code. Untuk setiap karakter, berdasarkan state DFA saat ini, lexer menentukan transisi ke state berikutnya atau menyelesaikan pembacaan satu token dan memancarkannya.

5. **Deteksi Error**. Jika lexer menemukan simbol atau urutan karakter yang tidak valid, lexer memancarkan token ERROR dengan pesan diagnostik, namun tetap melanjutkan proses tokenisasi (error recovery).

6. **Format Output**. Token-token yang telah dikumpulkan diformat menjadi string output sesuai dengan spesifikasi format lexer. Jika terdapat gap baris pada source code, baris kosong disisipkan pada output untuk mempertahankan struktur visual.

7. **Penulisan Output**. Output dicetak ke terminal (standard output). Jika nama file output diberikan sebagai argumen, output juga ditulis ke file tersebut.

8. **Kode Keluar**. Program keluar dengan exit code 0 jika tidak ada error leksikal atau exit code 1 jika terdapat error.

Secara arsitektur, program dibagi menjadi beberapa modul yang terpisah untuk meningkatkan modularitas dan maintainability. Modul `token` mendefinisikan enum TokenType yang berisi 52 jenis token plus ERROR, struktur Token untuk menyimpan informasi token (tipe, nilai, baris, kolom), dan fungsi-fungsi utilitas untuk konversi tipe ke string dan pemformatan output. Modul `lexer` mengimplementasikan class atau struktur Lexer yang mengenkapsulasi state DFA, buffer pembacaan, dan logika transisi. Modul `main` bertugas menangani argument parsing, file I/O, dan orchestration keseluruhan proses dari pembacaan input sampai penulisan output.

3.3 Algoritma Lexer

Algoritma lexer pada program ini menggunakan pendekatan *single pass scanning*, yaitu membaca source code sekali dari kiri ke kanan secara karakter demi karakter. Selama proses pembacaan, lexer menyimpan *state* aktif DFA, posisi karakter saat ini, serta buffer lexeme yang sedang dibangun. Setiap karakter yang dibaca akan memicu transisi ke state berikutnya, atau menandakan bahwa sebuah token telah lengkap dan siap dikeluarkan. Dengan pendekatan ini, kompleksitas waktu tokenisasi tetap linear terhadap panjang input, yakni $O(n)$, karena setiap karakter diproses dengan jumlah operasi konstan.

Secara lebih rinci, alur algoritma dimulai dari tahap validasi argumen masukan, kemudian pembacaan file `input.txt` ke dalam string `source`. Setelah itu, objek lexer diinisialisasi pada state awal `START` dengan posisi awal karakter di indeks pertama. Fungsi `tokenize()` lalu dijalankan dalam loop utama hingga akhir input. Pada setiap iterasi, handler state yang sesuai akan dieksekusi, misalnya handler untuk identifier, angka, string, operator, delimiter, atau komentar. Jika lexeme valid selesai dibaca, lexer memanggil `emitToken(...)`; jika ditemukan pola tidak valid, lexer memanggil `emitError(...)` namun tetap melanjutkan pemrosesan untuk mendukung *error recovery*.

Setelah seluruh input dipindai, program menjalankan `handleEof()` untuk menangani kasus token yang belum tertutup di akhir file (contohnya komentar atau string yang tidak ditutup). Daftar token yang dihasilkan kemudian diformat menjadi teks keluaran menggunakan fungsi `formatOutput(...)`. Hasil ini selalu ditulis ke terminal, dan jika argumen file output tersedia, hasil yang sama disimpan ke `output.txt`. Program mengembalikan exit code 0 jika tidak ada error leksikal, atau 1 jika ditemukan sedikitnya satu token error.

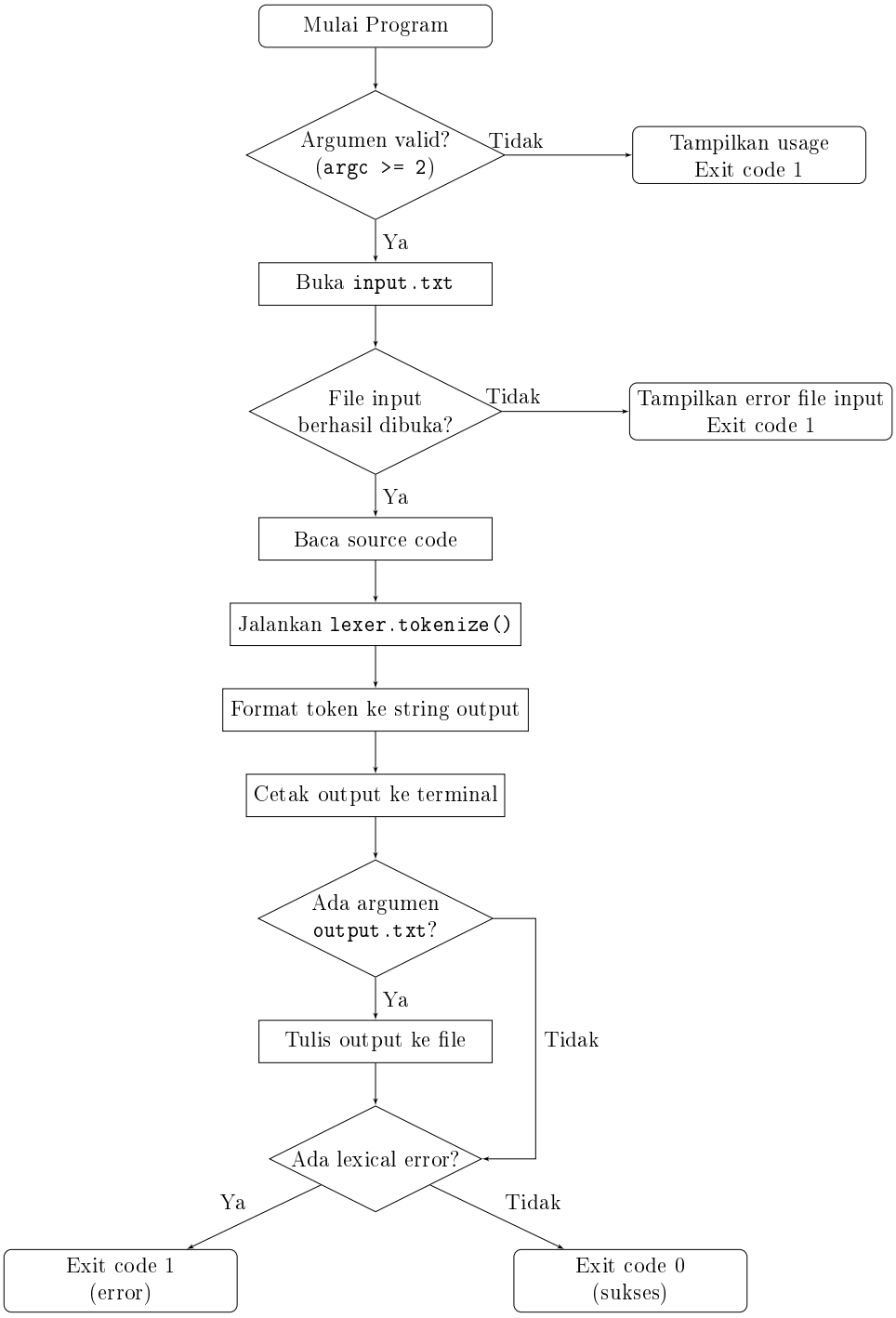
Pseudocode garis besar algoritma program ditunjukkan pada Algoritma 1.

Algorithm 1 Garis Besar Algoritma Lexer (`input.txt` → `output.txt`)

Require: Argumen command-line: `argv[1]` = `input.txt`, `argv[2]` (opsional) = `output.txt`
Ensure: Daftar token tercetak ke terminal dan opsional tersimpan ke `output.txt`

```
if jumlah argumen < 2 then
    Tampilkan pesan penggunaan program
    return exit code 1
end if
Buka berkas input.txt
if berkas gagal dibuka then
    Tampilkan pesan error file input
    return exit code 1
end if
Baca seluruh isi berkas ke string source
Inisialisasi lexer dengan source
tokens := lexer.tokenize()
output := formatOutput(tokens)
Cetak output ke terminal
if argumen output.txt diberikan then
    Buka/buat berkas output.txt
    if berkas output gagal dibuka then
        Tampilkan pesan error file output
        return exit code 1
    end if
    Tulis output ke output.txt
end if
if lexer.hasErrors() = true then
    return exit code 1
else
    return exit code 0
end if
```

Flowchart pada Gambar 3.2 memperlihatkan alur eksekusi program secara visual dari penerimaan argumen hingga penentuan exit code.



Gambar 3.2: Flowchart alur eksekusi program lexer dari input ke output

Lebih lanjut, proses `lexer.tokenize()` dapat diringkas sebagai berikut. Selama belum mencapai akhir input, lexer memeriksa state saat ini, menjalankan handler state yang sesuai, dan melakukan salah satu dari tiga aksi

1. `advance` ke karakter berikutnya,
2. `emit token` jika lexeme selesai, atau
3. `emit error` jika pola tidak valid.

Ketika loop selesai, `handleEof()` dipanggil untuk menyelesaikan state tertunda.

3.4 Struktur Direktori Program

- `src/`. Direktori utama yang berisi semua source code program dalam bahasa C++. Direktori ini dibagi menjadi dua modul:
  - `main.cpp`. File entry point program. Bertanggung jawab untuk parse argumen command-line, membuka dan membaca file input, menginisialisasi lexer, memanggil fungsi `tokenize()`, memformat output, dan menulis hasil ke terminal serta file output (jika ada). File ini juga menangani error handling untuk file I/O dan menentukan exit code berdasarkan ada tidaknya lexical error.
  - `lexer/`. Subdirektori yang berisi modul lexer dan token. Didalam subdirektori ini terdapat:
    * `token.h` dan `token.cpp`. File ini berisi modul yang mendefinisikan struktur atau class `Token`, enum `TokenType` dengan 52 jenis token plus ERROR, dan fungsi-fungsi utilitas seperti konversi tipe token ke string dan pemformatan output token.
    * `lexer.h` dan `lexer.cpp`. File ini berisi Modul yang mengimplementasikan class atau struktur `Lexer`. Lexer mengenkapsulasi state mesin DFA, buffer pembacaan source code, posisi karakter saat ini, dan logika-logika transisi state. Fungsi-fungsi utama meliputi `tokenize()` untuk loop tokenisasi, handler-handler state (misalnya handler identifier, angka, string, operator, komentar), dan fungsi `emitToken()/emitError()` untuk mengeluarkan token.
- `doc/`. Direktori dokumentasi proyek yang berisi laporan Milestone 1 dalam format LaTeX.
- `test/`. Direktori test case yang menyimpan input, expected output, dan actual output program untuk pengujian fungsionalitas lexer. Struktur di dalam direktori ini adalah sebagai berikut.
  - `milestone-1/`. Subdirektori untuk test case Milestone 1 yang berisi
    * `input-1.txt, input-2.txt, ..., input-N.txt`. File input berisi source code dalam bahasa Arion untuk di-test.
    * `expected-1.txt, expected-2.txt, ..., expected-N.txt`. File output yang diharapkan untuk setiap input (hasil tokenisasi yang benar).
    * `output-1.txt, output-2.txt, ..., output-N.txt`. File output aktual yang dihasilkan program ketika dijalankan terhadap input.
- `Makefile`. File build automation yang berisi rule-rule untuk kompilasi program.

- `README.md`. File dokumentasi yang berisi deskripsi singkat program, fitur-fitur, requirement sistem, instruksi build dan run, contoh input/output, struktur direktori, informasi penulis, dan lisensi MIT.

3.5 Implementasi Kode

3.5.1 `token.h`

File `token.h` mendefinisikan enum `TokenType` yang merepresentasikan 52 jenis token dalam bahasa Arion, ditambah satu token ERROR untuk penanganan kesalahan leksikal. File ini juga mendefinisikan struct `Token` yang menyimpan informasi lengkap tentang setiap token yang dihasilkan lexer, serta deklarasi fungsi-fungsi utilitas untuk konversi dan pemformatan.

```
enum class TokenType {
    // Literal
    INTCON,                 // Konstanta integer
    REALCON,                // Konstanta bilangan riil
    CHARCON,                // Konstanta karakter
    STRING,                 // Sekuens karakter

    // Operator
    PLUS, MINUS, TIMES,     // Aritmatika dasar
    IDIV, RDIV, IMOD,       // Pembagian dan modulo
    EQL, NEQ, GTR, GEQ,     // Perbandingan
    LSS, LEQ,

    // Keyword
    VARSY, PROGRAMSY, IFSY, BEGINSY, ENDSY,
    WHILESY, FORSY, // ... total 52 token

    // Error handling
    ERROR
};
```

Listing 3.1: Snippet enum TokenType dari token.h

Struktur Token. Struct `Token` adalah tipe data yang menyimpan informasi lengkap tentang token yang dihasilkan lexer:

```
struct Token {
    TokenType type;         // Jenis token (dari enum TokenType)
    std::string value;      // Nilai token untuk ident,
    //intcon, realcon, charcon, string
    int line;               // Nomor baris (1-based)
    int column;             // Nomor kolom (1-based)
};
```

Listing 3.2: Struktur Token dari token.h

Fungsi Utilitas. File ini juga mendeklarasikan tiga fungsi utilitas penting:

- `std::string tokenTypeToString(TokenType type)` . Fungsi ini digunkan untuk mengonversi enum TokenType menjadi string untuk ditampilkan pada output. Contohnya, `INTCON` dikonversi menjadi `"intcon"`.
- `bool tokenHasValue(TokenType type)`. Fungsi ini digunakan untuk mengecek apakah token memiliki nilai yang perlu ditampilkan. Contohnya adalah IDENT, INTCON, REALCON, CHARCON, STRING memiliki nilai, sedangkan PLUS, SEMICOLON tidak.
- `std::string formatToken(const Token& token)`. Fungsi ini digunakan untuk Memformat token untuk output sesuai spesifikasi format lexer. Jika token memiliki nilai, format output adalah `<tipe-token>(<nilai>)`, misalnya `ident(x)`; jika tidak ada nilai, hanya `<tipe-token>`, misalnya `plus`.

3.5.2 `token.cpp`

File `token.cpp` mengimplementasikan tiga fungsi utilitas yang dideklarasikan di `token.h`.

1. **tokenTypeToString()**. Fungsi ini mengkonversi nilai enum `TokenType` menjadi representasi string. Implementasinya menggunakan `switch-case` untuk memetakan setiap token type ke string yang sesuai, semua dalam huruf kecil sesuai format output yang ditentukan. Contohnya, `TokenType::INTCON` dikonversi menjadi `"intcon"`, `TokenType::PROGRAMSY` menjadi `"programsy"`, dan seterusnya.
2. **tokenHasValue()**. Fungsi ini mengecek apakah suatu tipe token memiliki nilai yang perlu ditampilkan dalam tanda kurung. Token dengan nilai antara lain IDENT (identifier), INTCON (konstanta integer), REALCON (konstanta riil), CHARCON (konstanta karakter), STRING (string literal), COMMENT (komentar), dan ERROR (pesan error). Sementara kategori token lain seperti operator, delimiter, dan keyword tidak memiliki nilai yang ditampilkan.
3. **formatToken()**. Fungsi ini memformat token untuk output. Jika token memiliki nilai (berdasarkan `tokenHasValue()`), format outputnya adalah `<tipe>(<nilai>)`, misalnya `ident(Hello)` atau `intcon(42)`. Jika token tidak memiliki nilai, format outputnya hanya `<tipe>`, misalnya `plus` atau `semicolon`.

Snippet dari `token.cpp` ditunjukkan berikut.

```
std::string formatToken(const Token& token) {
    if (tokenHasValue(token.type)) {
        return tokenTypeToString(token.type) + " (" +
               token.value + ")";
    }
    return tokenTypeToString(token.type);
}
```

Listing 3.3: Implementasi formatToken() dari token.cpp

3.5.3 `lexer.h`

File `lexer.h` mendeklarasikan enum `State` yang merepresentasikan 13 state dalam DFA lexer, serta kelas `Lexer` yang mengimplementasikan mesin DFA.

Enum State. Enum ini mendefinisikan semua state yang ada dalam DFA sebagai berikut.

- `START`. State awal, menunggu karakter baru untuk menentukan jenis token berikutnya.
- `IN_IDENT`. Sedang membaca sebuah identifier atau keyword (karakter alfanumerik dan underscore).
- `IN_NUMBER`. Sedang membaca digit-digit untuk konstanta integer.
- `DOT_AFTER_NUM`. Baru saja membaca '.', yang bisa merupakan period (delimiter) atau bagian dari bilangan riil.
- `IN_REAL`. Sedang membaca bagian desimal dari bilangan riil.
- `IN_STRING`. Sedang membaca karakter di dalam tanda kutip untuk string literal.
- `SAW_COLON`, `SAW_LESS`, `SAW_GREATER`, `SAW_EQUAL`. State yang melihat operator karakter tunggal yang mungkin dilanjutkan dengan karakter lain (untuk membentuk operator multi-karakter seperti `:=`, `<>`, `>=`, `==`).
- `IN_COMMENT`. Sedang membaca karakter di dalam komentar (baik `{ ...}` maupun `(* ...*)`).
- `SAW_LPAREN`. Melihat '(', yang bisa merupakan left parenthesis atau awal komentar `(*...*)`.
- `SAW_STAR_IN_COMMENT`. Di dalam komentar dan melihat '*', yang mungkin merupakan akhir komentar `*)` .

Kelas Lexer. Kelas ini mengenkapsulasi semua data dan logika untuk melakukan tokenisasi. Data member privat meliputi data sebagai berikut.

- `source`. String berisi seluruh source code yang akan di-tokenisasi.
- `pos, line, col`. Posisi pembacaan saat ini (posisi karakter, nomor baris, nomor kolom).
- `state`. State DFA saat ini.
- `buffer`. Akumulasi karakter untuk token yang sedang diproses.
- `tokenStartLine, tokenStartCol`. Posisi awal token yang sedang diproses (untuk keperluan error reporting).
- `errorFlag`. Flag untuk menandai adanya error leksikal.

- `tokens`. Daftar (vector) hasil token yang dihasilkan.

Fungsi publik meliputi fungsi-fungsi di bawah ini.

- `Lexer(const std::string& source)`. Konstruktor yang menginisialisasi lexer dengan source code.
- `std::vector<Token> tokenize()`. Fungsi utama yang menjalankan DFA dan mengembalikan daftar token hasil analisis.
- `bool hasErrors() const`. Mengecek apakah ada error leksikal.

Fungsi privat meliputi handler untuk setiap state (`handleStart()`, `handleInIdent()`, `handleInNumber()`, dll.), serta fungsi-fungsi utilitas seperti `emitToken()`, `emitError()`, dan `lookupKeyword()`.

3.5.4 lexer.cpp

File `lexer.cpp` mengimplementasikan kelas `Lexer`, termasuk fungsi utilitas privat dan handler untuk setiap state DFA. Implementasi mengikuti pola *state machine* klasik. Polanya adalah untuk setiap state, handler yang sesuai membaca karakter saat ini, memutuskan tindakan dan melanjutkan ke karakter berikutnya.

Tabel Keyword. Di awal file terdapat `static const std::map<>` yang menyimpan 27 keyword dari bahasa Arion. Perbandingan dilakukan case-insensitive dengan mengkonversi identifier ke lowercase sebelum lookup. Contohnya, identifier `PROGRAM` akan dicocokkan dengan key `program` dan menghasilkan token `PROGRAMSY`.

Konstruktor dan Fungsi Utilitas. Konstruktor `Lexer()` menginisialisasi semua member variable. Fungsi utilitas `current()`, `isAtEnd()`, dan `advance()` menangani pembacaan karakter dan update posisi line/column. Fungsi `emitToken()` dan `emitError()` menambahkan token/error ke daftar output. Fungsi `lookupKeyword()` mencari identifier di tabel keyword.

Fungsi tokenize() Utama. Fungsi ini menjalankan loop utama yang membaca hingga akhir input, memanggil handler state yang sesuai, dan setelah akhir file, memanggil `handleEof()` untuk menyelesaikan token yang belum selesai. Detail dari `tokenize()` dapat dilihat di Snippet kode di bawah ini.

```
std::vector<Token> Lexer::tokenize() {
    while (!isAtEnd()) {
        switch (state) {
            case State::START:      handleStart(); break;
            case State::IN_IDENT:   handleInIdent(); break;
            case State::IN_NUMBER:  handleInNumber(); break;
            // ... handler untuk state lainya ...
        }
    }
    handleEof();  // Tangani token yang belum selesai
    return tokens;
}
```

Listing 3.4: Loop utama tokenize() dari lexer.cpp

Handler State. Setiap state memiliki handler yang bertanggung jawab untuk logika transisi DFA. Contohnya:

- **handleStart()**. Handler ini Membaca karakter pertama dan menentukan jenis token. Jika huruf, transisi ke `IN_IDENT`. Jika digit, transisi ke `IN_NUMBER`. Jika tanda kutip, transisi ke `IN_STRING`. Operator dan delimiter single-character langsung emit token. Whitespace diabaikan.
- **handleInIdent()**. Handler ini akan mengakumulasi huruf/digit ke buffer. Ketika melihat non-alphanumeric, lookup keyword dan emit token IDENT atau keyword, lalu kembali ke `START`.
- **handleInNumber()**. Handler ini akan mengakumulasi digit. Ketika melihat '.', transisi ke `DOT_AFTER_NUM` untuk menentukan apakah itu bilangan riil atau period. Non-digit menyebabkan emit INTCON dan kembali ke `START`.
- **handleDotAfterNum()**. Jika digit berikutnya ada, gabungkan ke buffer dan transisi ke `IN_REAL`. Jika bukan digit, emit INTCON, emit PERIOD, dan kembali ke `START`.
- **handleSawColon(), handleSawLess(), dst.**. Handler ini untuk operator multi-karakter. Misalnya, setelah `:` dilihat, jika karakter berikutnya adalah `=`, emit BECOMES; sebaliknya emit COLON.
- **handleInString()**. Akumulasi karakter dalam tanda kutip. Jika menemukan tanda kutip penutup, tentukan apakah string (panjang > 1) atau CHARCON (panjang $= 1$).
- **handleInComment()**. Akumulasi karakter dalam comment. Komentar gaya `\{ \}` ditutup dengan `\}`, sedangkan `(* \))` ditutup dengan `*)` (menggunakan state `SAW_STAR_IN_COMMENT`).

Contoh snippet dari `handleStart()` ditunjukkan sebagai berikut, memperlihatkan bagaimana handler menentukan transisi berdasarkan karakter saat ini.

```
void Lexer::handleStart() {
    char c = current();

    if (std::isspace(c)) {
        advance();  // Abaikan whitespace
    } else if (std::isalpha(c)) {
        buffer = c;
        state = State::IN_IDENT;  // Mulai baca identifier
        advance();
    } else if (std::isdigit(c)) {
        buffer = c;
        state = State::IN_NUMBER;  // Mulai baca bilangan
        advance();
    } else if (c == '+') {
        emitToken(TokenType::PLUS);
```

```
        advance();
    } else if (c == ':') {
        state = State::SAW_COLON;  // Bisa : atau :=
        advance();
    } else {
        // Karakter tidak dikenali
        emitError("Unexpected character");
        advance();
    }
}
```

Listing 3.5: Snippet handleStart() dari lexer.cpp

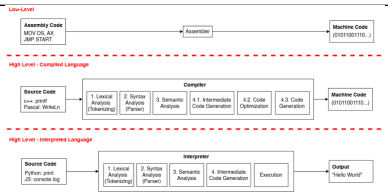
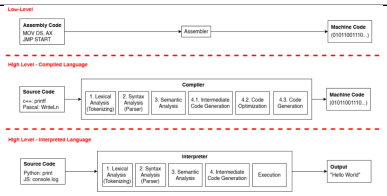
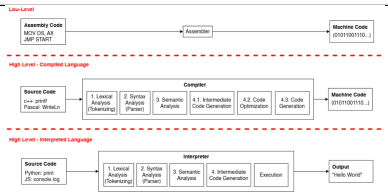
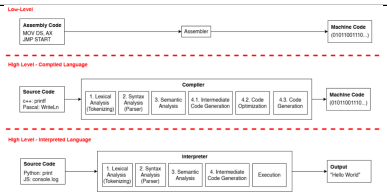
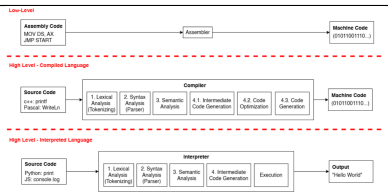
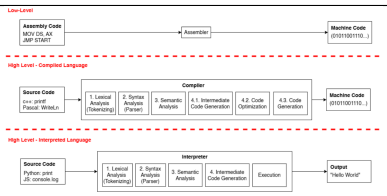
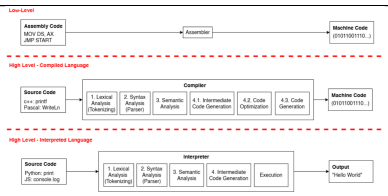
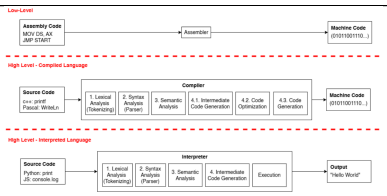
Dengan struktur *state machine* ini, lexer dapat mengenali semua pola token dalam bahasa Arion dengan cara yang terstruktur dan mudah dipahami. Setiap transisi state mewakili suatu tahap dalam proses pengenalan token, dan handler memastikan akumulasi buffer serta penentuan token type yang tepat sesuai pola yang ditemukan.

Bab 4

Pengujian

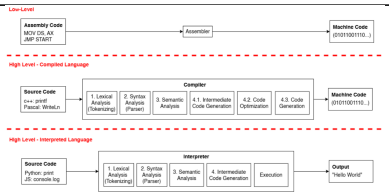
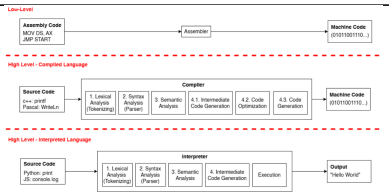
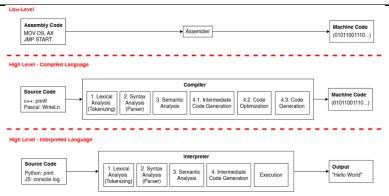
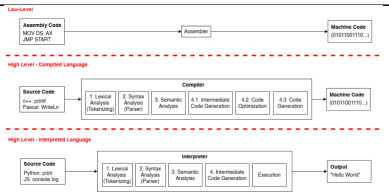
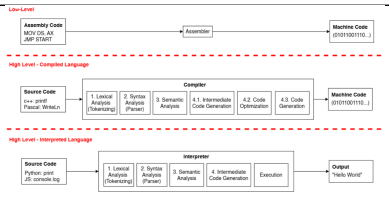
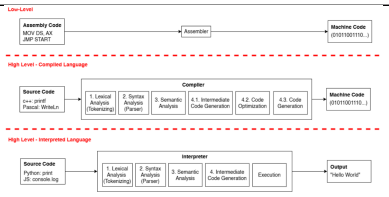
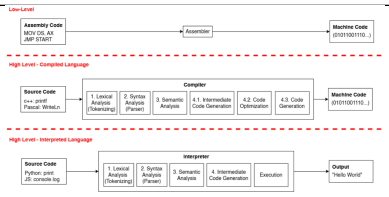
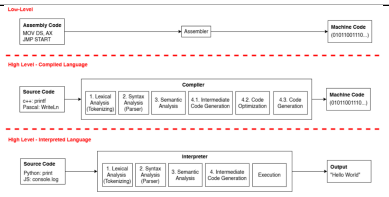
4.1 Kasus Dasar

Tabel 4.1: Hasil Pengujian pada Kasus Dasar

No	Test Case	Hasil
1		
2		
3		
4		

4.2 *Edge Cases*

Tabel 4.2: Hasil Pengujian pada Kasus Anomali

No	Test Case	Hasil
1		
2		
3		
4		

Bab 5

Penutup

5.1 Kesimpulan

5.2 Saran

Bab 6

Lampiran

6.1 Tautan GitHub

Link GitHub

6.2 Tautan Diagram Transisi DFA

6.3 Tabel Kontribusi

No	Nama Lengkap	NIM	Deskripsi Pekerjaan	Persentase Kontribusi (%)
1	Jonathan Kris Wicaksono	13524023	Memrogram konsep	25
2	Vincent Rionarlie	13524031	Melakukan pembagian tugas, mendesain diagram transisi DFA	25
3	Muhammad Aufar Rizqi Kusuam	13524061	Menyusun laporan akhir	25
4	Bryan Pratama Putra Hendra	13524067		25

Bibliografi

[1] J. E. Hopcroft, R. Motwani, and J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed. Pearson, 2006.

[2] M. Sipser, *Introduction to the Theory of Computation*, 3rd ed. Cengage Learning, 2012.

[3] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Addison-Wesley, 2007.

[4] D. Grune, K. van Reeuwijk, H. E. Bal, C. J. H. Jacobs, and K. Langendoen, *Modern Compiler Design*, 2nd ed. Springer, 2012.